MyResume - Phase 2: Infrastructure as Code (IaC) with Terraform

Overview

My Resume is an online, cloud-deployed version of my portfolio. This project focuses on the practical application of Microsoft Azure services and demonstrating foundational cloud architecture and deployment skills. This project is developed as the kickstart for my Cloud Computing journey.

Phase 2 of this project involves the IaC deployment of Phase 1, where after developing the different stacks locally, I proceeded to code the terraform files to provision the different necessary resources for the project and running them.

This document will serve as the overview for MyResume Phase 2: IaC with Terraform.

The Tech Stack:

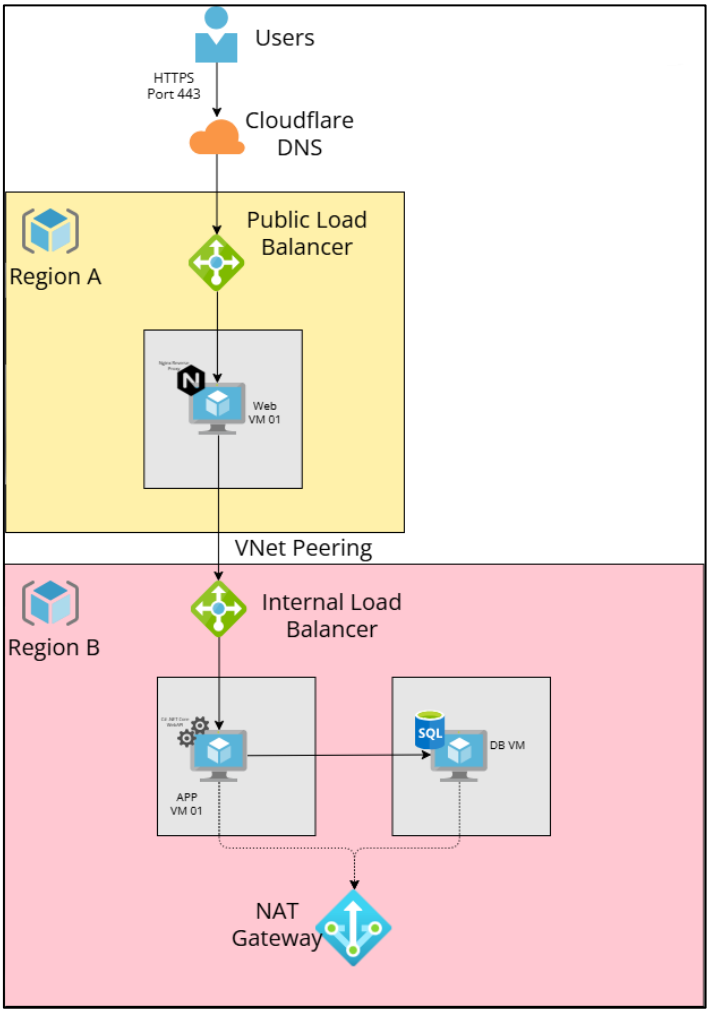
- Frontend: React with Typescript (Hosted via Nginx in the Virtual Machine)
- Backend: C# .NET Core Web API (Ran as service with Kestrel)
- Database: Microsoft SQL Server

Contents

Architecture

I will be implementing the same architecture planned for Phase 1:

- 2 Separate regions (1 for the frontend & 1 for the backend)
- 1 vm for each layer
- 1 Public & 1 Internal Load Balancer (To mimic actual production environment)



Azure Resources

The list of resources provisioned are in 2 Resource Groups:

- rg-myresume-prod (Region A):
  - vnet-myresume-prod
  - lb-myresume-app
  - nat-myresume-egress
  - vm-myresume-app-01 (with nsg, disks)
  - vm-myresume-db-01 (with nsg, disks)
- rg-myresume-web-prod (Region B):
  - vnet-myresume-web-prod
  - lb-myresume-web
  - vm-myresume-web-01 (with public IP, nsg, disks)

Terraform

The Terraform infrastructure is decoupled into four dedicated modules managed by a root:

- Storage
- Frontend
- Backend

Storage Module is where the:

- Storage Account
- Blob Storages for the React & API files
- Shared Access Signature (SAS)

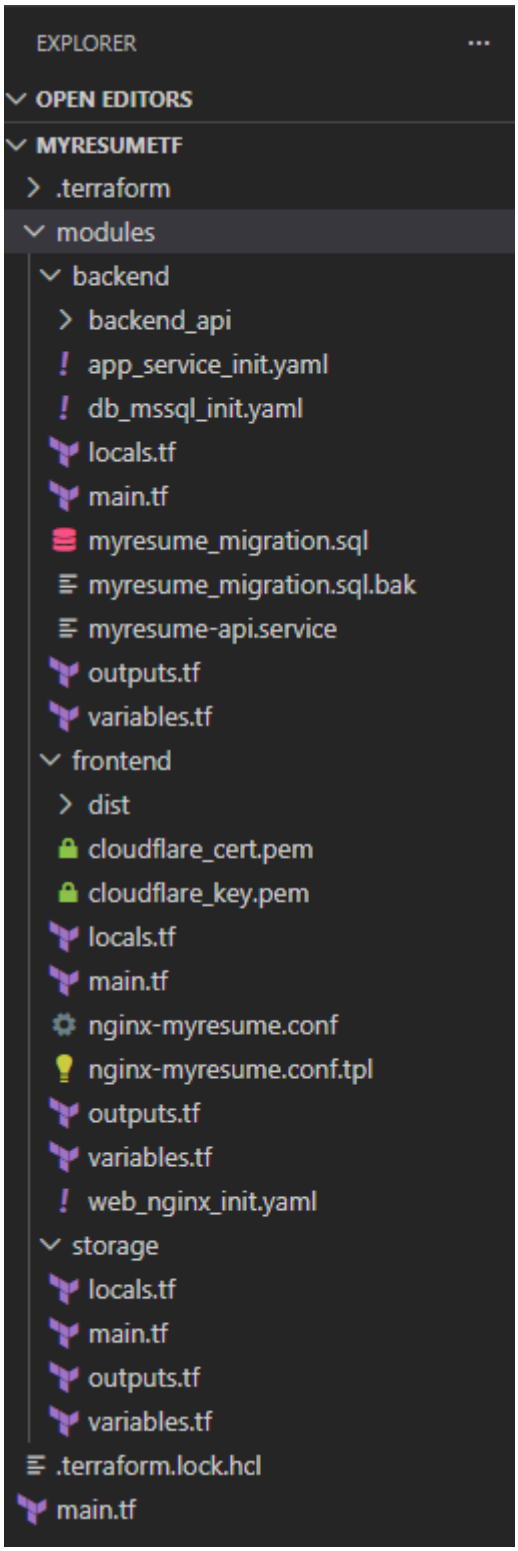
Are provisioned.

These are needed so that Terraform is able to upload the project files and bring up Nginx and service in the web and app vms respectively.

Frontend Module is where the resources related to the web app (Region A) are provisioned.

Backend Module is where the resources related to the API Service & Database (Region B) are provisioned.

I referenced HashiCorp and Microsoft Learn pages for the configuration of each resources.



```
C:\MyResumeTF>terraform init
Initializing modules...
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/azurerm from the dependency lock file
- Reusing previous version of hashicorp/archive from the dependency lock file
- Reusing previous version of hashicorp/random from the dependency lock file
- Reusing previous version of hashicorp/local from the dependency lock file
- Reusing previous version of hashicorp/tls from the dependency lock file
- Using previously-installed hashicorp/local v2.9.0
- Using previously-installed hashicorp/tls v4.3.0
- Using previously-installed hashicorp/azurerm v3.0.2
- Using previously-installed hashicorp/archive v2.8.0
- Using previously-installed hashicorp/random v3.9.0

Initializing HCP Terraform...

Initializing provider plugins found in the state...


HCP Terraform has been successfully initialized!

You may now begin working with HCP Terraform. Try running "terraform plan" to
see any changes that are required for your infrastructure.

If you ever set or change modules or Terraform Settings, run "terraform init"
again to reinitialize your working directory.
```

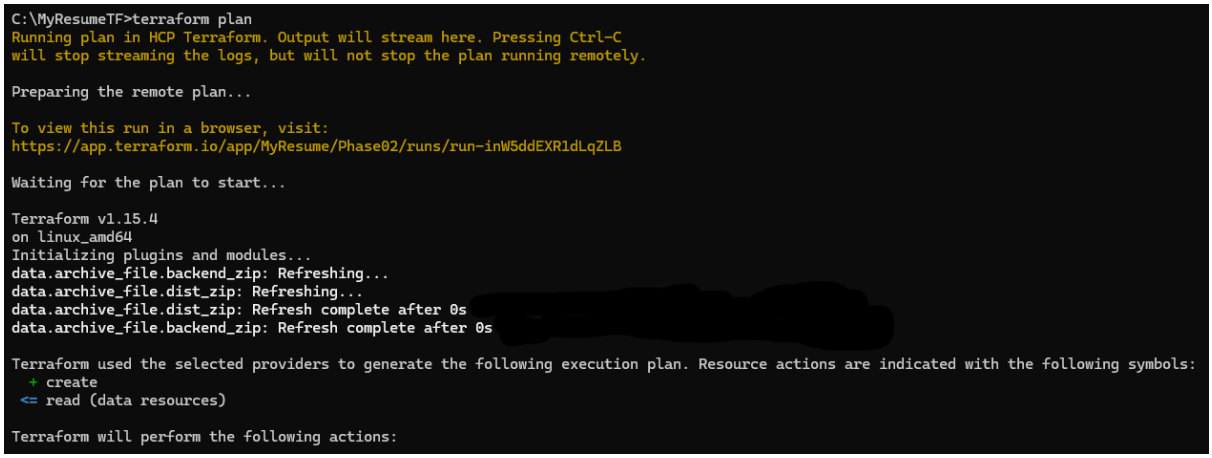
terraform init:

This command initialises the working directory. It downloads/updates the necessary provider plugins (in this project, Azure) and configures the backend storage for the state file. The state file is a JSON file that acts as a source of truth for the managed infrastructure (what is provisioned, what needs to be provisioned/destroyed/replaced). This command needs to be run at the start of the project and is safe to run at any point of time in development.

```
C:\MyResumeTF>terraform validate
Success! The configuration is valid.
```

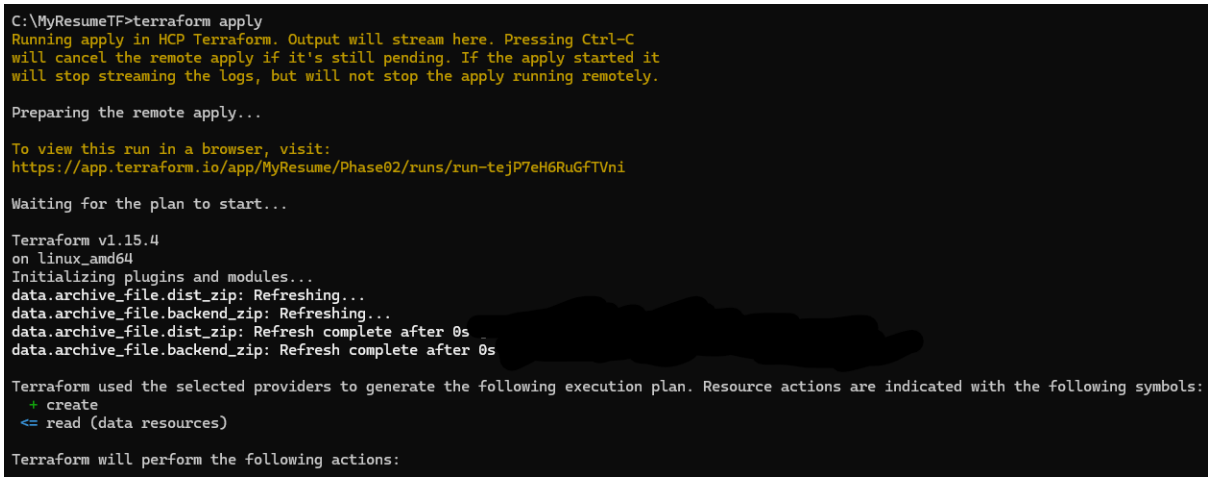
terraform validate:

This command checks the configuration codes for syntax error and internal consistency. If there are errors, the message will be in reed and points to the specific file and line number. If there are none, it will print out Success in green. Think of this as the Error/Warning list in IDEs like Visual Studio. It is a quick way to ensure the terraform code is valid before attempting to deploy.



terraform plan:

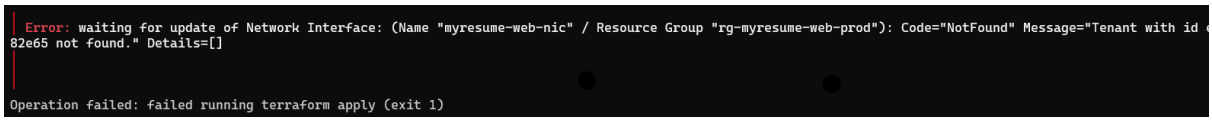
This command compares the Terraform code against the managed infrastructure in the state file. It generates an execution plan showing exactly what resources will be provisioned/replaced/destroyed. After the texts ‘Terraform will perform the following actions:’, the terminal will show a detailed list of the outcome of each resources.



terraform apply:

This command executes the execution plan generated by ‘terraform plan’. It applies the changes to Azure to build/update/destroy the managed infrastructure. The terminal will once again show in detail the list of outcomes for each resources. If the flag ‘-auto-approve’ is not given along with the command, the user will be prompted to type ‘yes’ to confirm the deployment.

Once done, the terminal will show the results, if something wrong happens along the way an error message will be printed and terraform will stop the process and update the state file.



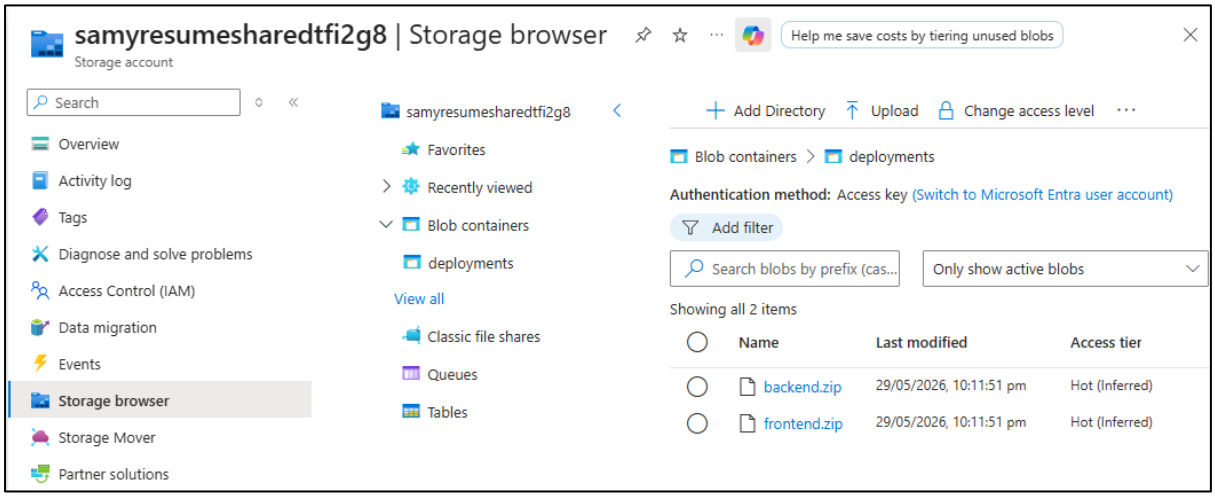
```
C:\MyResumeTF>terraform destroy -auto-approve
Running apply in HCP Terraform. Output will stream here. Pressing Ctrl-C
will cancel the remote apply if it's still pending. If the apply started it
will stop streaming the logs, but will not stop the apply running remotely.
```

terraform destroy:

This command is the opposite of terraform apply, it permanently terminates and deprovisions all infrastructure managed by the Terraform configuration. If the flag '-auto-approve' is not given along with the command, the cmd will prompt to type in 'yes' to continue.

Storage Layer

The Storage Module provisions an Azure Storage Account and stages the compiled application binaries as secure blobs. The root module captures these output URLs to inject them directly into the compute layer deployment scripts. ‘frontend.zip’ is the built React code and ‘backend.zip’ is the published C# .NET WebAPI code.



Frontend Layer

Tied directly to the Phase 1 networking layout, the frontend module now encapsulates the Public Load Balancer and the Nginx Web VM. On boot, cloud-init processes web_init.yaml to install Nginx and automatically configure the reverse-proxy routing rules.

modules > frontend > ! web_nginx_init.yaml

```yaml
#cloud-config
package_update: true
packages:
  - nginx
  - unzip

write_files:
  - path: /etc/nginx/sites-available/resume
    content: |
      ${indent(6, nginx_config)}
    permissions: '0644'

  - path: /etc/ssl/certs/cloudflare_cert.pem
    content: |
      ${indent(6, ssl_cert)}
    permissions: '0644'

  - path: /etc/ssl/private/cloudflare_key.pem
    content: |
      ${indent(6, ssl_key)}
    permissions: '0600'

runcmd:
  - wget -O /tmp/dist.zip "${download_url}"

  - mkdir -p /var/www/html
  - unzip -o /tmp/dist.zip -d /var/www/html/

  - ln -sf /etc/nginx/sites-available/resume /etc/nginx/sites-enabled/
  - rm -f /etc/nginx/sites-enabled/default

  - chown -R www-data:www-data /var/www/html
  - systemctl enable nginx
  - systemctl restart nginx

  - rm -f /tmp/dist.zip
```

modules > frontend > nginx-myresume.conf.tpl

```
server {
    listen 80;
    listen [::]:80;
    server_name faliqdoescoding.com www.faliqdoescoding.com;

    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    listen [::]:443 ssl http2;
    server_name faliqdoescoding.com www.faliqdoescoding.com localhost;

    ssl_certificate     /etc/ssl/certs/cloudflare_cert.pem;
    ssl_certificate_key /etc/ssl/private/cloudflare_key.pem;

    ssl_protocols       TLSv1.2 TLSv1.3;
    ssl_ciphers         HIGH:!aNULL:!MD5;
    ssl_prefer_server_ciphers on;

    root /var/www/html;
    index index.html index.htm;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://${internal_lb_ip}:5000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
        proxy_set_header X-Forwarded-Port 443;
    }
}
```

```
azureuser@myresume-web-vm:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-05-29 14:12:51 UTC; 58min ago
       Docs: man:nginx(8)
    Process: 2341 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Process: 2343 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 2344 (nginx)
      Tasks: 3 (limit: 9492)
     Memory: 4.2M (peak: 4.5M)
        CPU: 211ms
     CGroup: /system.slice/nginx.service
             ├─2344 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             ├─2345 "nginx: worker process"
             └─2346 "nginx: worker process"

May 29 14:12:51 myresume-web-vm systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
May 29 14:12:51 myresume-web-vm systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
azureuser@myresume-web-vm:~$
```

A change in the React Code was to stitch together all the portfolio information into one interface and perform only one API call to get the portfolio data.

```tsx
import type { CertificationProps } from "./CertificationProps";
import type { EducationProps } from "./EducationProps";
import type { ExperienceProps } from "./ExperienceProps";
import type { FileProps } from "./FileProps";
import type { ImageProps } from "./ImageProps";
import type { InformationProps } from "./InformationProps";
import type { ProjectProps } from "./ProjectProps";

export interface PortfolioProps {
    information: InformationProps
    experiences: ExperienceProps[]
    educations: EducationProps[]
    certifications: CertificationProps[]
    projects: ProjectProps[]
    files: FileProps[]
    imageDetails: ImageProps[]
}
```

```tsx
import api from "./API";
import type { PortfolioProps } from "../types/PortfolioProps";

export class PortfolioService {
    static async getPortfolio(): Promise<PortfolioProps> {
        try {
            const response = await api.get('Portfolio');
            return response.data as PortfolioProps;
        } catch (error) {
            console.error("Error fetching portfolio:", error);
            throw error;
        }
    }
}

export default PortfolioService;
```

Backend Layer

The backend module contains API Service VM and the MSSQL VM. On boot, cloud-init processes app_service_init.yaml and db_mssql_init.yaml to install the necessary .NET runtimes, create the API service, install mssql and migrate the live data into the database.

```
azureuser@myresume-app-vm:~$ cat /etc/systemd/system/myresume-api.service
[Unit]
Description=C# Resume API Application
After=network.target

[Service]
WorkingDirectory=/var/www/api
ExecStart=/usr/bin/dotnet /var/www/api/MyResumeBackend.dll --urls "http://0.0.0.0:5000"
Restart=always
RestartSec=10
KillSignal=SIGINT
SyslogIdentifier=myresume-api
User=azureuser
Environment=ASPNETCORE_ENVIRONMENT=Production
Environment=DOTNET_PRINT_TELEMETRY_MESSAGE=false
Environment=ASPNETCORE_URLS=http://0.0.0.0:5000

Environment="ConnectionStrings__DefaultConnection=Server=10.2.4.4;Database=my_resume_db;User Id=sa;Password=[redacted];TrustServerCertificate=True;"
Environment="Cors__AllowedOrigins=https://faliqdoescoding.com,https://www.faliqdoescoding.com"

[Install]
WantedBy=multi-user.target
```

```
azureuser@myresume-app-vm:~$ systemctl status myresume-api --no-pager
● myresume-api.service - C# Resume API Application
     Loaded: loaded (/etc/systemd/system/myresume-api.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-05-29 14:13:36 UTC; 37min ago
   Main PID: 7191 (dotnet)
      Tasks: 19 (limit: 9505)
     Memory: 41.3M (peak: 42.8M)
        CPU: 1.975s
     CGroup: /system.slice/myresume-api.service
             └─7191 /usr/bin/dotnet /var/www/api/MyResumeBackend.dll --urls http://0.0.0.0:5000

May 29 14:13:36 myresume-app-vm systemd[1]: Started myresume-api.service - C# Resume API Application.
May 29 14:13:37 myresume-app-vm myresume-api[7191]: info: Microsoft.Hosting.Lifetime[14]
May 29 14:13:37 myresume-app-vm myresume-api[7191]:       Now listening on: http://0.0.0.0:5000
May 29 14:13:37 myresume-app-vm myresume-api[7191]: info: Microsoft.Hosting.Lifetime[0]
May 29 14:13:37 myresume-app-vm myresume-api[7191]:       Application started. Press Ctrl+C to shut down.
May 29 14:13:37 myresume-app-vm myresume-api[7191]: info: Microsoft.Hosting.Lifetime[0]
May 29 14:13:37 myresume-app-vm myresume-api[7191]:       Hosting environment: Production
May 29 14:13:37 myresume-app-vm myresume-api[7191]: info: Microsoft.Hosting.Lifetime[0]
May 29 14:13:37 myresume-app-vm myresume-api[7191]:       Content root path: /var/www/api
azureuser@myresume-app-vm:~$
```

```
azureuser@myresume-db-vm:~$ sudo systemctl status mssql-server
● mssql-server.service - Microsoft SQL Server Database Engine
     Loaded: loaded (/usr/lib/systemd/system/mssql-server.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-05-29 14:14:16 UTC; 49min ago
       Docs: https://docs.microsoft.com/en-us/sql/linux
   Main PID: 2848 (sqlservr)
      Tasks: 81
     Memory: 676.3M (peak: 696.1M)
        CPU: 1min 27.577s
     CGroup: /system.slice/mssql-server.service
             ├─2848 /opt/mssql/bin/sqlservr
             └─2939 /opt/mssql/bin/sqlservr

May 29 14:14:37 myresume-db-vm sqlservr[2939]: [128B blob data]
May 29 14:14:37 myresume-db-vm sqlservr[2939]: [115B blob data]
May 29 14:14:37 myresume-db-vm sqlservr[2939]: [119B blob data]
May 29 14:14:37 myresume-db-vm sqlservr[2939]: [125B blob data]
May 29 14:14:38 myresume-db-vm sqlservr[2939]: [106B blob data]
May 29 14:14:38 myresume-db-vm sqlservr[2939]: [114B blob data]
May 29 14:14:39 myresume-db-vm sqlservr[2939]: [105B blob data]
May 29 14:19:36 myresume-db-vm sqlservr[2939]: [156B blob data]
May 29 14:19:36 myresume-db-vm sqlservr[2939]: [195B blob data]
May 29 14:21:24 myresume-db-vm sqlservr[2939]: [71B blob data]
```

```
azureuser@myresume-db-vm:~$ sqlcmd -S localhost -U sa -P [redacted] -C
1> USE my_resume_db
2> GO
Changed database context to 'my_resume_db'.
1> SELECT COUNT(*) FROM files
2> SELECT COUNT(*) FROM projects
3> GO

-----------
         38

(1 rows affected)

-----------
          2

(1 rows affected)
1>
```

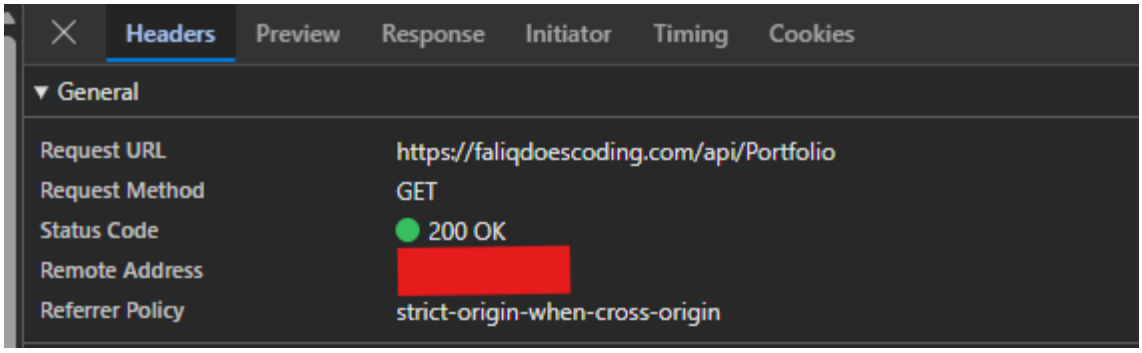
A change in the code for this Phase to prepare for Phase 3 was to have only one controller endpoint where the HTTP Get request will hit to return the stitched data of the whole portfolio.

```
[HttpGet]

public async Task<IActionResult> GetPortfolio()
{
    try
    {
        var portfolio = await _projectService.GetPortfolio();
        if (portfolio == null)
            return NotFound();
        return Ok(portfolio);
    }
    catch (Exception ex)
    {
        return StatusCode(StatusCodes.Status500InternalServerError, "An error occurred while retrieving portfolio. " + ex);
    }
}
```

```
public class PortfolioDTO
{

    public InformationDTO information { get; set; }

    public IEnumerable<ExperienceDTO> experiences { get; set; }

    public IEnumerable<EducationDTO> educations { get; set; }
    1 reference
    public IEnumerable<CertificationDTO> certifications { get; set; }
    1 reference
    public IEnumerable<ProjectDTO> projects { get; set; }
    1 reference
    public IEnumerable<FileDTO> files { get; set; }
    1 reference
    public IEnumerable<ImageDTO> imageDetails { get; set; }
}
```

Verification & Results

A live handshake confirmation demonstrating successful end-to-end data flow from the database to the public edge can be viewed in the networks tab.



Conclusion

This phase marked the transition from manual ‘ClickOps’ to Infrastructure as Code (IaC) using Terraform. By codifying the infrastructure, the process ensured that resource provisioning became consistent, repeatable and version-controlled. This directly improves deployment reliability.